Networking Essentials

網路是系統設計的基礎——你幾乎永遠都在設計由獨立裝置組成、透過網路互相溝通的系統。但網路這個領域既廣闊又複雜，很容易迷失在細節裡。

這篇講義會聚焦在系統設計面試中你真正需要知道的網路知識。我們從網路運作的基礎開始，然後檢視網路堆疊不同層次的關鍵協定，對每個概念說明它的用途、運作方式，以及什麼時候應該在設計中運用它。

> **給不同背景讀者的提示**：網路在基礎設施和分散式系統面試中是重點。對於全端和產品導向的職位，你可能只需要對網路概念有表面上的理解。就算細節不會被考，理解這些基礎仍然能幫助你做出更好的設計決策。
>
> 每個面試官都不一樣——如果你的面試官剛從一次牽涉到負載平衡器（Load Balancer）或內容傳遞網路（CDN） 問題的 oncall 值班結束，你最好準備好回應他的追問！

網路 101

網路的核心就是連接裝置並讓它們能夠溝通。網路建立在一個分層架構上——也就是大家說的「OSI 模型」——這大大簡化了我們作為應用程式開發者需要理解的世界。

網路層次本質上就是一層層的抽象，讓我們能用和應用程式相關的簡單語言來思考裝置之間的通訊。這樣一來，當你請求一個網頁時，你不需要知道網路線上哪個電壓代表 0 或 1——你只需要知道怎麼使用堆疊中的下一層。就像你在程式語言裡用 `open` 開檔案，而不需要親自指揮磁碟怎麼讀取 bytes 一樣。

網路層次

雖然完整的網路堆疊非常迷人，但在系統設計面試中最常出現的只有三個關鍵層次。讓我們先了解它們分別做什麼、又如何協作。

網路層（Network Layer） （Layer 3）

這一層是 IP 協定，負責路由和定址。它把資料拆分成封包（packet）、處理封包在網路之間的轉送，並提供盡力而為（best-effort）的交付服務，把封包送往網路上任意的目的地 IP 位址。雖然這一層還有其他協定（例如用於大型機器學習訓練工作負載的 InfiniBand），但 IP 是系統設計面試裡最常遇到的。

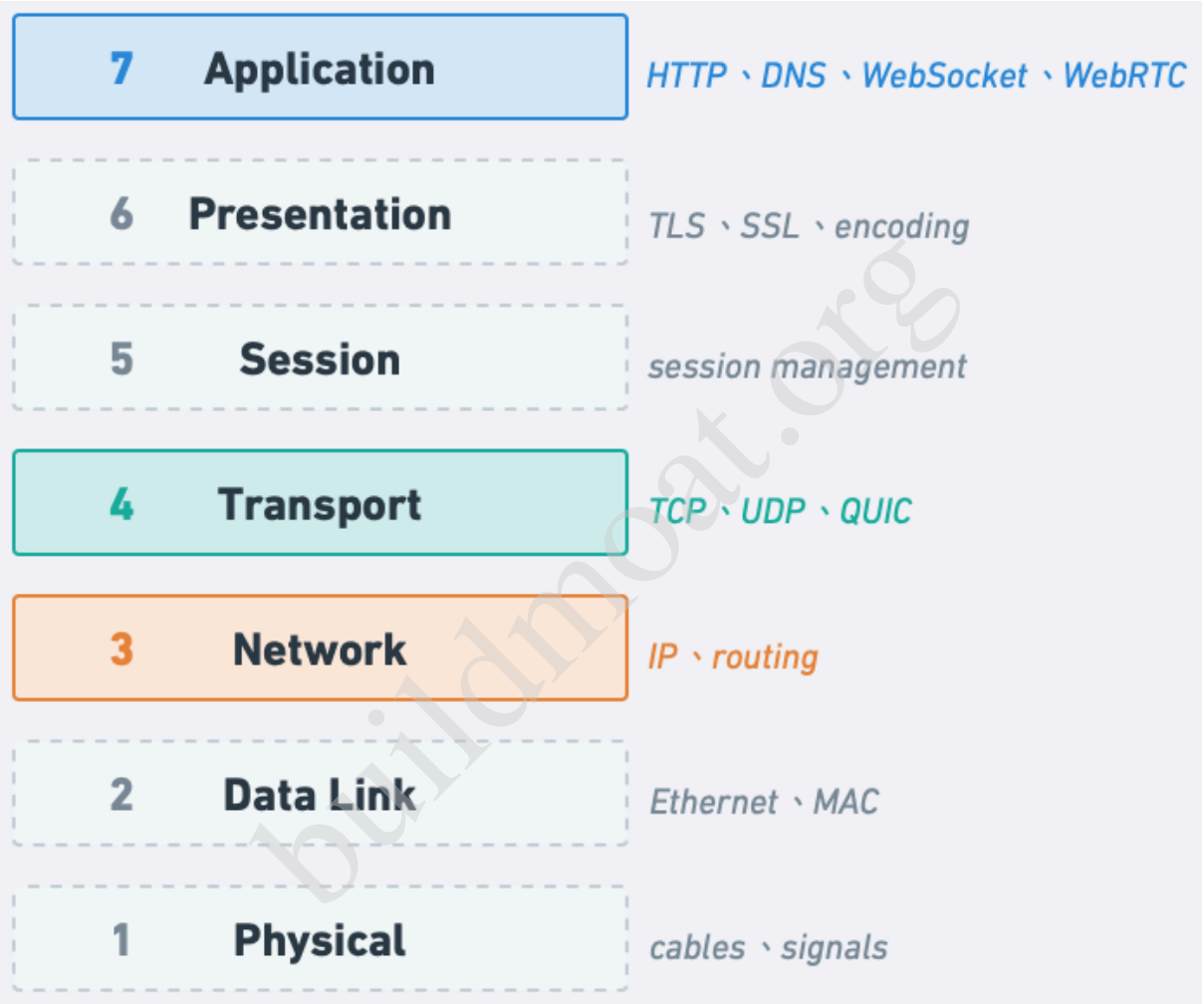
傳輸層 （Transport Layer ） （Layer 4）

buildmoat.org

這一層有 TCP、QUIC 和 UDP，提供端到端的通訊服務。你可以把它們想成在網路層之上增加了可靠性、排序、流量控制等功能的一層。

應用層 （Application Layer） （Layer 7）

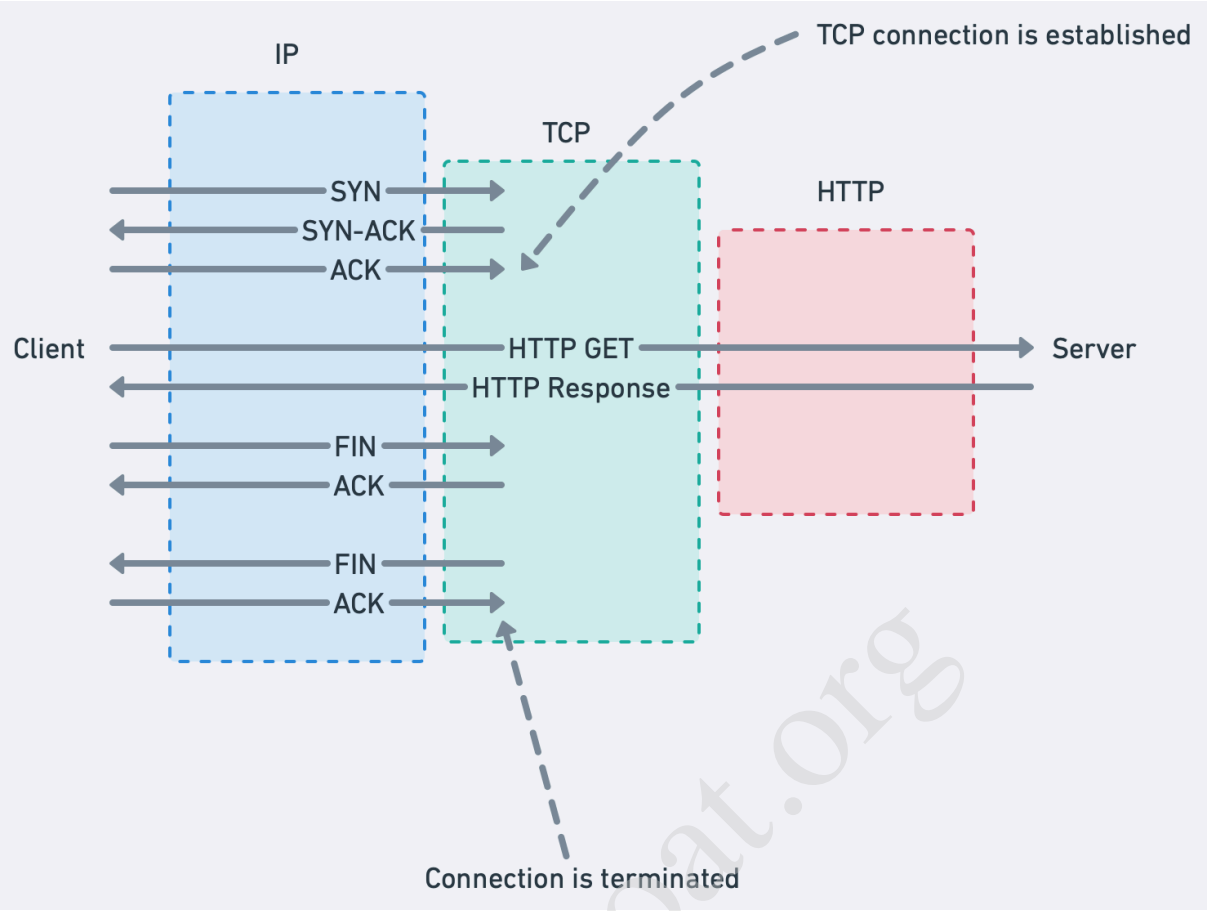
最頂層是應用程式協定，例如 DNS、HTTP、WebSocket、WebRTC。這些常見的協定建立在 TCP（或 UDP，以 WebRTC 為例）之上，為通常和 Web 應用相關的不同類型資料提供抽象層。



例子：一個簡單的 Web 請求

當你在瀏覽器輸入一個 URL 時，幾個網路協定層次會同時動起來。讓我們看看它們是如何協作，透過 HTTP over TCP 取得一個簡單的網頁的。

首先，我們用 DNS 把人類可讀的域名（例如 https://www.buildmoat.org）轉換成 IP 位址（例如 32.42.52.62）。然後，一系列精心編排的步驟開始了：建立 TCP 連線、發送 HTTP 請求、取得回應、拆除連線。



詳細流程：

1. **DNS 解析**：客戶端用 DNS（Domain Name System）把網站的域名解析成 IP 位址。
2. **TCP 三次握手（Three-Way Handshake）**：
   - **SYN**：客戶端發送 SYN（同步）封包給伺服器，請求建立連線
   - **SYN-ACK**：伺服器回應 SYN-ACK（同步確認）封包，確認請求
   - **ACK**：客戶端發送 ACK（確認）封包，完成連線建立
3. **HTTP 請求**：TCP 連線建立後，客戶端向伺服器發送 HTTP GET 請求。
4. **伺服器處理**：伺服器處理請求、取得網頁、準備 HTTP 回應。（這通常是大多數軟體工程師唯一會想到並能控制的延遲！）
5. **HTTP 回應**：伺服器把包含網頁內容的 HTTP 回應發回客戶端。
6. **TCP 四次揮手（Four-Way Teardown）**：
   - **FIN**：客戶端發送 FIN（結束）封包，通知終止連線
   - **ACK**：伺服器確認 FIN 封包

buildmoat.org

- **FIN**：伺服器發送 FIN 封包，終止它這端的連線
- **ACK**：客戶端確認伺服器的 FIN 封包

「在瀏覽器裡輸入https://www.buildmoat.org並按下 Enter 會發生什麼事？」曾經是 BigTech 面試中的經典問題，近年較少見，但理解這些基礎對實際工作很有幫助。

這個例子中有幾件值得注意的事：

第一，身為應用程式開發者，我們可以大幅簡化心智模型。應用程式可以理所當然地假設資料以一定的可靠性和順序傳輸——TCP 層確保資料正確且有序地送達，如果失敗了也會告知應用程式。我們也不需要操心怎麼找到世界某處的特定伺服器。有了 DNS 和 IP，各種網路硬體和服務商會負責把資料路由到目的地。

第二，雖然概念上只有一個「請求」和「回應」，但背後交換的封包和請求遠不止這些。這些都帶來我們可以忽略的延遲……直到忽略不了為止。越往堆疊上層走，需要的延遲和處理就越多——這在我們後面討論負載平衡器時很重要！

第三，客戶端和伺服器之間的連線是一個雙方都必須維護的狀態。除非使用 HTTP keep-alive 或 HTTP/2 多路複用（multiplexing）等功能，否則每個請求都需要重複這個連線建立過程——這是相當可觀的額外開銷。在設計需要持久連線的系統（例如即時更新）時，這一點至關重要。

傳輸層協定

傳輸層是我們建立應用程式之間端到端通訊的地方。這一層的三個主要協定是 TCP、UDP 和 QUIC，各自有不同的特性，適合不同的使用場景。

在大多數系統設計面試中，你真正需要做的選擇是 TCP 和 UDP 之間的取捨。QUIC 是一個新的協定，目標是提供 TCP 的部分優勢並加入現代化的效能改進。雖然 QUIC 越來越流行，但它相對還是新的協定，尚未廣泛普及——對我們的目的來說，可以把它視為更好版本的 TCP，只是沒有相同廣泛的基準支援。

> 一些注重效能的面試官會對你了解 QUIC 和 HTTP/3 這樣的現代協定印象深刻，但大多數系統設計面試官希望你把時間花在設計的其他地方。

UDP：快速但不可靠

UDP（User Datagram Protocol）是協定中的機關槍。它在 IP 之上幾乎沒有任何額外功能，但非常快速。「向前衝、聽天命」是理解它的正確方式。它提供一個簡單的、無連線的服務，不保證送達、不保證順序、不保護重複。

如果你寫了一個接收 UDP 資料報的應用程式，你能知道它從哪裡來（來源 IP 和 port）和要去哪裡（目的 IP 和 port）。就這樣，剩下的就是一個二進位 blob。

UDP 的關鍵特性：

buildmoat.org

- **無連線**：不需要握手或連線建立
- **不保證送達**：封包可能在沒有任何通知的情況下遺失
- **不保證順序**：封包可能以和發送不同的順序到達
- **更低的延遲**：較少的額外開銷意味著更快的傳輸

沒有連線建立聽起來很好，但第 2 和 3 點挺糟的，所以為什麼要用 UDP？

UDP 非常適合速度比可靠性更重要的應用，例如直播影片串流、線上遊戲、VoIP（網路電話）和 DNS 查找。在這些情況下，應用程式或客戶端有能力自行處理偶爾的封包遺失或亂序。以 VoIP 為例，客戶端可能只是丟棄偶爾遺失的封包，導致音訊有一點卡頓，但整體對話仍然清晰。這遠比重新傳輸遺失的封包、用 ACK 塞爆網路要好得多。

注意：目前除了 WebRTC 之外，瀏覽器對 UDP 的支援並不廣泛。如果你在設計可能用到 UDP 的方案（例如 Facebook 直播留言裡的愛心和表情），要想好瀏覽器用戶怎麼辦。可能的方案是：用原生 App 的用戶獲得真正的 UDP 即時串流，而瀏覽器用戶則使用較慢的批次 HTTP 串流，在 UI 上分散顯示。

TCP：可靠但有額外開銷

TCP（Transmission Control Protocol）是網際網路的主力。它提供可靠的、有序的、有錯誤檢查的資料交付。它透過三次握手建立連線（就像我們在 HTTP 例子裡看到的），並在整個通訊期間維護這個連線。

這個連線叫做「串流（stream）」，是客戶端和伺服器之間的有狀態連線——它也給了我們討論順序的基礎：在同一個串流/連線中發送的兩則訊息，會以相同的順序到達。TCP 會確保訊息的接收方確認收到，如果他們沒有確認，就會重新傳輸，直到被確認為止。

TCP 的關鍵特性：

- **面向連線**：在資料傳輸前建立專用的連線
- **可靠交付**：保證資料有序且無錯誤地到達
- **流量控制**：防止傳送方以超過接收方處理能力的速率發送資料
- **壅塞控制**：適應網路壅塞，防止網路崩潰

TCP 非常適合資料完整性至關重要的應用——也就是幾乎所有 UDP 不適合的場景。

什麼時候選哪個協定

在系統設計面試中，大多數面試官預設你使用 TCP——通常不需要特別說明。這也是我們的建議！

buildmeat.org

但如果你能為使用 UDP 提出合理的論據而不搞砸細節，就能額外加分。所以你應該問自己的問題是：UDP 是否更適合你的使用場景？

考慮使用 UDP 的情況：

- 低延遲至關重要（即時應用、遊戲）
- 一些資料遺失是可以接受的（媒體串流）
- 處理高流量的遙測或日誌，偶爾的遺失可以接受
- 不需要支援網頁瀏覽器（或者有針對瀏覽器的替代方案）

現代應用往往同時使用兩種協定。例如，一個基於網頁的視訊會議應用可能用 TCP/HTTP 做信號傳遞和驗證，但用 UDP/WebRTC 做實際的音視訊串流。

特性	UDP	TCP
連線方式	無連線	面向連線
可靠性	盡力而為交付	保證交付
順序	不保證順序	維護順序
流量控制	無	有
壅塞控制	無	有
標頭大小	8 bytes	20-60 bytes
速度	較快	因額外開銷而較慢
使用場景	串流、遊戲、VoIP	其他所有情況

buildroat.org

應用層協定

應用層是大多數開發者花費最多時間的地方。這些協定定義了應用程式如何通訊，建立在我們剛討論的傳輸層協定之上。

HTTP/HTTPS：網路的基礎

HTTP（Hypertext Transfer Protocol）是網路上資料通訊的事實標準。它是一個請求-回應協定，客戶端向伺服器發送請求，伺服器回應所請求的資料。

HTTP 是一個**無狀態**協定，意思是每個請求都是獨立的，伺服器不需要維護任何關於先前請求的資訊。這通常是件好事——在系統設計中，你希望盡可能縮小系統中需要有狀態的部分。大多數簡單的 HTTP 伺服器可以被描述為請求參數的函式——它們是無狀態的！

```
# A simple http request
GET /api/users/1 HTTP/1.1          # method
Host: example.com                  #
Accept: application/json           # headers
Authorization: Bearer abc123token  #
```

```
# A simple http response
HTTP/1.1 200 OK.                     # status code
Content-Type: application/json       #
Date: Mon, 23 Feb 2026 10:00:00 GMT # headers
Content-Length: 89                   #

{
  "id": 1,
  "name": "Terry Chen",
  "email": "terry@example.com",      # body
  "role": "admin"
}
```

一個 HTTP 請求/回應包含幾個關鍵概念：

- **請求方法**：GET、POST、PUT、DELETE 等
- **狀態碼**：200 OK、404 Not Found、500 Server Error 等
- **Headers**：關於請求或回應的 metadata
- **Body**：實際傳輸的內容

常見的請求方法：

- `GET`：從伺服器請求資料。GET 請求應該是冪等的，沒有 body。
- `POST`：向伺服器發送資料。
- `PUT`：更新伺服器上的資料。
- `PATCH`：部分更新一個資源。
- `DELETE`：從伺服器刪除資料。DELETE 請求應該是冪等的。

常見的狀態碼：

成功（2xx）：`200 OK`（請求成功）、`201 Created`（請求成功且建立了新資源）

buildmoat.org

重定向（3xx）：`302 Found`（資源暫時移動）、`301 Moved Permanently`（資源永久移動）

客戶端錯誤（4xx）：`404 Not Found`（資源不存在）、`401 Unauthorized`（需要驗證）、`403 Forbidden`（理解請求但拒絕授權）、`429 Too Many Requests`（請求頻率過高）

伺服器錯誤（5xx）：`500 Server Error`（伺服器遇到錯誤）、`502 Bad Gateway`（從上游伺服器收到無效回應）

Headers 要靈活得多（想成鍵值對）。HTTP 的 `Accept-Encoding` header 是個很好的例子：客戶端可以表明它能處理不同類型的內容編碼，伺服器就可以用最有效率的編碼（例如 gzip 或 brotli）回應，同時保持向後相容性。這是一個值得學習的 API 設計哲學。

HTTPS 在 HTTP 之上加了一個安全層（TLS/SSL）來加密通訊，防止竊聽和中間人攻擊。如果你在建立一個公開的網站，毫無例外都要用 HTTPS。

> **重要安全提醒**：雖然 HTTPS 的請求和回應內容是加密的，但這並不保證它們是由你的客戶端產生的！你的 API 在沒有驗證的情況下，永遠不應該信任請求 body 的內容。一個常見的錯誤是在請求 body 裡包含用戶 ID，然後用它來做資料庫查詢。如果攻擊者能修改請求 body，他們就能修改用戶 ID，讀取任意用戶的資料。

REST：簡單且靈活

雖然 HTTP 可以直接用來建立網站，但系統設計通常關注的是服務之間透過 API 的通訊。建立這些 API 有三個主要的範式：REST、GraphQL 和 gRPC。

REST 是你在系統設計面試中最常用的 API 範式。核心原則是客戶端通常對「資源（resource）」執行簡單的操作（把資源想成資料庫表格或伺服器上的檔案）。

RESTful API 設計利用 HTTP 方法（動詞）加上關於路徑和請求 body 的一些慣例。通常用 JSON 表示請求和回應 body 中的資源。

如果你遵循系統設計的交付框架，你的核心實體（Core Entities）通常會直接對應到 API 裡的資源。

一個簡單的 RESTful API 例子（其中 User 是代表用戶的 JSON 物件）：

```
GET    /users/{id}       → User          取得特定用戶
PUT    /users/{id}       → User          更新用戶資料
POST   /users            → User          建立新用戶
GET    /users/{id}/posts → [Post]        取得用戶的所有貼文
```

buildmoai.org

```
POST /users
{
  "username": "bohr",
  "email": "bohr@buildmoat.org"
}
```

很多工程師習慣以 `updateUser` 或 `startGame` 這樣的方法來思考。這些是操作（operation），不是資源，所以不是 RESTful 的設計。在 REST 中，我們要以資源和你可以對它們執行的操作來思考：`updateUser` 變成 `PUT /users/{id}`，`startGame` 變成 `PATCH /games` 加上 `{ "status": "started" }`。

什麼時候用：REST 對各種使用場景都非常靈活。就像 TCP 一樣，REST 是面試的預設推薦選擇。除非有 REST 無法滿足的特定需求，才考慮 GraphQL、gRPC、SSE 或 WebSocket。

GraphQL：彈性的資料獲取

GraphQL 是一個較新的 API 範式（約 2015 年由 Facebook 開源），讓客戶端能夠請求它精確需要的資料。

GraphQL 解決的問題是：前端和後端往往分屬不同的團隊。當前端團隊想顯示一個新頁面，他們要嘛（a）拼湊一堆不同的後端端點請求（想像為了獲取 10 個用戶的詳情而發出 10 個 API 呼叫），要嘛（b）建立龐大的聚合 API（難以維護且難以變更），要嘛（c）為每個新頁面寫全新的 API。這些都不是特別好的解法。

- 欠獲取（Under-fetching）的問題是你可能需要多個請求和往返。這增加了額外開銷和頁面載入的延遲。
- 過獲取（Over-fetching）是相反的情況：為了應付未來可能有的使用案例，我們在 API 回應裡塞進了遠多於當下所需的資料。API 載入慢且回傳了太多資料。

GraphQL 解決了這些問題，讓前端團隊能靈活地向後端查詢它精確需要的資料，後端則以前端需要的形狀回應資料。

一個 GraphQL 查詢的例子，取得顯示複雜頁面所需的用戶資料（包含個人資料和群組）：

```
query GetUsersWithProfilesAndGroups($limit: Int = 10, $offs
et: Int = 0) {
  users(limit: $limit, offset: $offset) {
    id
    username
```

buildmoat.org

```
    profile {
      id
      fullName
      avatar
    }

    groups {
      id
      name
      description

      category {
        id
        name
        icon
      }
    }

    status {
      isActive
      lastActiveAt
    }
  }

  _metadata {
    totalCount
    hasNextPage
  }
}
```

什麼時候用：GraphQL 非常適合前端團隊需要快速迭代、靈活調整的使用場景，以及多個團隊對重疊資料進行廣泛查詢的情境。

在系統設計面試中，GraphQL 的好處其實比較模糊。面試裡的需求是固定的（不像在迭代行動 App 或 Web 前端時那樣會移動），而且 GraphQL 的後端執行有時會引入它本來試圖避免的那種複雜後端程式碼。我們建議只在問題明顯聚焦於靈活性時才提出 GraphQL。

buildmoat.org

gRPC：強調效能與低延遲的通訊協定

gRPC 是 Google 推出的高效能 RPC（遠端程序呼叫）框架，使用 HTTP/2 和 Protocol Buffers。

Protocol Buffers 就像 JSON，但有更嚴格的綱要，帶來更好的效能和更有效率的序列化。一個用戶資源的 Protocol Buffer 定義例子：

```
message User {
  string id = 1;
  string name = 2;
}
```

不像帶有嵌入式綱要的 JSON（40 bytes）：

```
{"id": "123", "name": "Terry Chen"}
```

……我們有了相同資料的二進位編碼（15 bytes），使用非常精簡的標籤和可變長度字串編碼。佔用空間更少，解析所需的 CPU 也更少！

gRPC 在此基礎上提供服務定義：

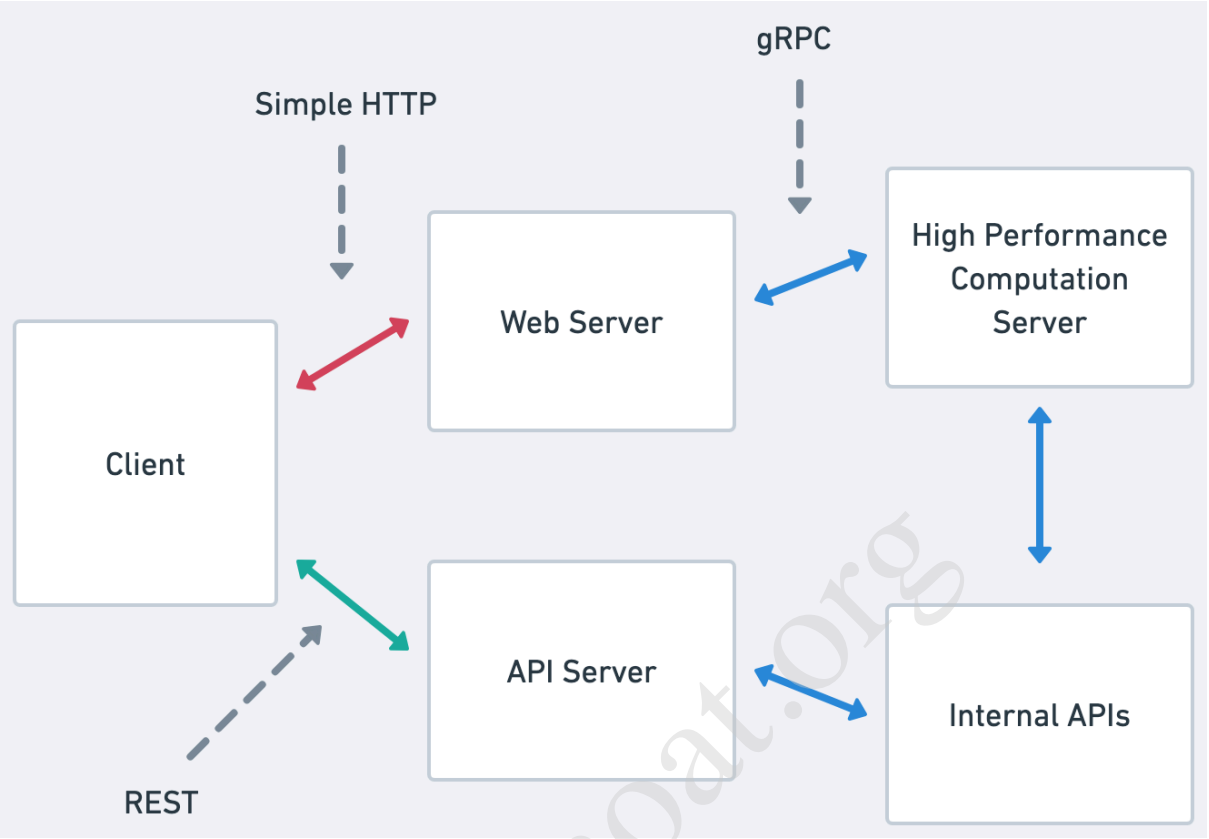
```
message GetUserRequest {
  string id = 1;
}

message GetUserResponse {
  User user = 1;
}

service UserService {
  rpc GetUser (GetUserRequest) returns (GetUserResponse);
}
```

這些定義被編譯成客戶端和伺服器 stub，多種語言和框架都可以用來建構服務和客戶端。gRPC 還包含許多對大規模運營微服務架構有用的功能（畢竟是 Google 發明的），例如串流、截止時間、客戶端負載平衡等。但最重要的是：它是一個比 HTTP over JSON 更快更有效率的二進位協定。

什麼時候用：gRPC 在微服務架構中的服務間高效通訊上表現出色。強型別有助於在編譯時而不是執行時發現錯誤，二進位協定比 JSON over HTTP 更有效率（某些基準測

buildmoat.org

試顯示吞吐量高達 10 倍！）。



不過，你通常不會用 gRPC 做公開的 API，尤其是你無法控制的客戶端，因為它是二進位協定，工具生態系統不如簡單的 JSON over HTTP 成熟。**一個好的策略是：內部 API 用 gRPC，外部 API 用 REST**——既獲得二進位協定的好處，又不增加公開 API 的複雜度。

> **面試提示**：不要過早優化 RPC 協定選擇。面試的目的是了解你如何思考問題，在解決其他實質性瓶頸之前就跳到 gRPC 並不是好的做法。過早的優化是萬惡之源！

SSE（Server-Sent Events）：即時伺服器推送

到目前為止我們討論的主要是請求/回應風格的 API，但許多應用程式需要以更串流的方式把資料「推送」給客戶端。SSE 是建立在 HTTP 之上的規範，讓伺服器能透過單一 HTTP 連線向客戶端推送多則訊息。

理解 SSE 的方式是：它是 HTTP 之上的一個巧妙改裝，讓伺服器能在單一回應中、隨時間持續串流多則訊息。

一般的 HTTP API 回應是一個完整的 JSON blob，需要等整個回應收完才能處理：

buildmoat.org

```
{
  "events": [
    {"id": 1, "timestamp": "2025-01-01T00:00:00Z", "descrip
tion": "Event 1"},
    {"id": 2, "timestamp": "2025-01-01T00:00:01Z", "descrip
tion": "Event 2"},
    ...
  ]
}
```

而用 SSE，伺服器可以在單一回應中以「區塊」的形式推送多則訊息：

```
data: {"id": 1, "timestamp": "2025-01-01T00:00:00Z", "descr
iption": "Event 1"}

data: {"id": 2, "timestamp": "2025-01-01T00:00:01Z", "descr
iption": "Event 2"}

...

data: {"id": 100, "timestamp": "2025-01-01T00:00:10Z", "des
cription": "Event 100"}
```

每一行作為一則獨立的訊息被接收，客戶端可以在它到達時立即處理。它仍然是一個 HTTP 回應（同一個 TCP 連線），但分成很多小封包進來，客戶端逐行處理 body，以便對資料做即時反應。

SSE 有一些固有的限制：SSE 連線不能保持太長時間，因為伺服器（或負載平衡器、中間代理）會關閉連線。SSE 標準定義了 `EventSource` 物件的行為——連線關閉後，它會自動用最後收到的訊息 ID 重新連線。伺服器需要追蹤客戶端斷線期間可能錯過的訊息並重新發送。

另外，某些不規範的網路可能會把所有 SSE 回應批次聚合成單一回應，讓它的行為退化成我們想要避免的那種。

大多數面試官不熟悉這些限制，會讓你假設它們不存在。但了解它們是好的，因為任何真正實作過 SSE 的人都對這些問題有深刻的記憶。

什麼時候用：在系統設計面試中，當你希望客戶端一有通知或事件就立刻收到時，SSE 是個好選擇。例如讓出價者即時看到拍賣當前價格。

buildmoat.org

WebSocket：即時雙向通訊

雖然 SSE 是伺服器向客戶端推送的好方案，但很多應用需要**雙向**的即時通訊。WebSocket 提供了客戶端和伺服器之間持久的、類似 TCP 的連線，實現即時的雙向通訊，並有廣泛的支援（包括瀏覽器）。不像 HTTP 的請求-回應模型，WebSocket 讓伺服器能在沒有新請求觸發的情況下主動推送資料給客戶端，客戶端也能隨時推資料給伺服器。

WebSocket 透過 HTTP「升級（upgrade）」協定發起，讓現有的 TCP 連線切換 L7 協定。這非常方便，因為你可以利用現有的 HTTP 工作階段資訊（例如 cookies、headers 等）。

> **注意**：客戶端能從 HTTP 升級到 WebSocket 並不代表基礎設施一定支援它。客戶端和伺服器之間的每一個基礎設施元件都需要支援 WebSocket 連線。如果你實作過 WebSocket，可能曾遇到過防火牆、代理、負載平衡器等不支援 WebSocket 連線的問題。

運作方式：

1. 客戶端透過 HTTP 發起 WebSocket 握手（底層是 TCP 連線）
2. 連線升級到 WebSocket 協定，WebSocket 接管 TCP 連線
3. 客戶端和伺服器都可以透過連線互相發送二進位訊息
4. 連線保持開啟，直到明確關閉為止

WebSocket 不規定應用協定，你基本上有一個可以從客戶端或伺服器發送二進位封包的通道。這意味著你需要定義客戶端和伺服器交換的是什麼。對很多 WebSocket 應用來說，簡單的序列化 JSON 訊息是個好選擇。

什麼時候用：WebSocket 在面試中出現，是在你需要客戶端和伺服器之間高頻、持久、雙向通訊的時候。考慮即時應用、遊戲，以及你需要一有訊息就立刻發送和接收的場景。

> **面試警告**：在沒有說明原因的情況下就跳進 WebSocket 實作，是讓面試官給你「thumbs down」的好方法。WebSocket 雖然強大，但支援它所需的基礎設施可能很昂貴，有狀態連線的開銷（尤其是在規模下）也需要設計上的大量配合。除非真的需要，否則不要用！

WebRTC：點對點通訊

最後一個協定是最獨特的。WebRTC 讓瀏覽器之間能夠直接進行點對點（peer-to-peer）通訊，不需要中間伺服器來做資料交換。WebRTC 非常適合視訊/音訊通話和會

bulunoat.org

議應用，也可以用於協作應用（如文件編輯器）。而且它是我們會介紹的唯一一個使用 UDP 的應用層協定！

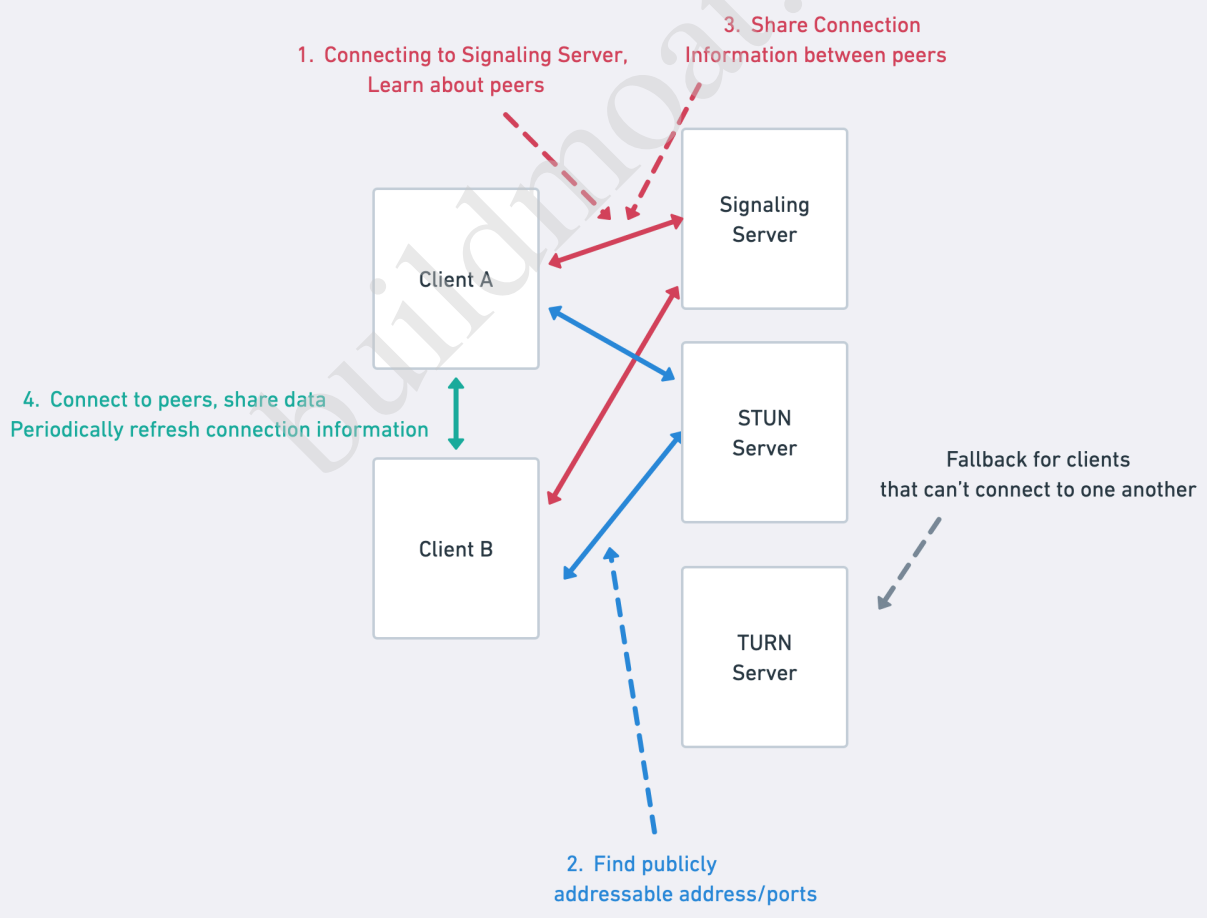
從網路的角度來看，點對點連線比我們一直討論的客戶端-伺服器模型更複雜，因為大多數客戶端出於安全原因不允許傳入連線。

透過 WebRTC，客戶端和一個中央「信令伺服器（signaling server）」溝通，信令伺服器追蹤哪些 peer 可用以及它們的連線資訊。一旦客戶端有了另一個 peer 的連線資訊，就可以嘗試建立直接連線，不需要透過任何中間伺服器。

但實際上，大多數客戶端都在 NAT（網路位址轉換）裝置後面，無法被直接連線。WebRTC 標準包含兩種解法：

STUN（Session Traversal Utilities for NAT）：一個協定和一套技術（如「打洞（hole punching）」），讓 peer 能建立可公開路由的位址和 port。它透過反覆建立開放的 port 並透過信令伺服器與 peer 分享，是處理 NAT 穿透的標準方式。

TURN（Traversal Using Relays around NAT）：實際上就是一個中繼服務，把請求透過中央伺服器轉送，再路由到目標 peer。



buildnoat.org

建立 WebRTC 連線的四個步驟：

1. 客戶端連線到中央信令伺服器，了解自己的 peer
2. 客戶端聯繫 STUN 伺服器取得自己的公網 IP 和 port
3. 客戶端透過信令伺服器與對方分享這些資訊
4. 客戶端建立直接的點對點連線，開始傳送資料

這是理想情況！現實中，有時連線會失敗，需要 TURN 伺服器等備案。

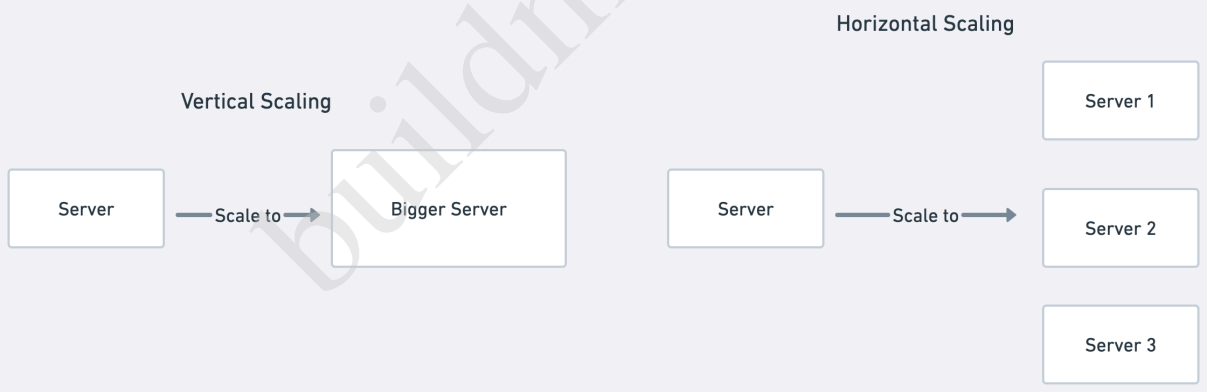
什麼時候用：WebRTC 非常適合音視訊通話和會議應用。在面試中，建議只在視訊/音訊通話和會議場景使用 WebRTC。

> **誠實的警告**：WebRTC 極難做對，即使是最好的實作也仍然會有連線中斷的問題。它真的是一個小眾解法。在面試中，嘗試用 WebRTC 設計點對點系統而搞錯方向的面試者，比成功實作的要多得多。大多數問題不需要點對點連線，很容易試圖把解法套在一個根本不需要它的問題上。

負載平衡 (Load Balancing)

我們已經涵蓋了整個堆疊的頂端和所有在系統設計面試中會看到的相關協定。但我們如何擴展設計呢？當然這裡也有網路的影響！

擴展有兩個選擇：**更大的伺服器（垂直擴展）或更多的伺服器（水平擴展）**。



在現在(2020 -)的年代，實際上是盡可能採用垂直擴展——現代硬體非常強大，需要幾千台小伺服器才能處理幾台大伺服器能承受的負載的時代已經過去了。

但面試的現實是，你最常看到的擴展模式是水平擴展：我們要加更多伺服器來處理負載。但光在白板上畫更多方塊沒有用，如果我們不告訴客戶端要跟哪台伺服器溝通。

這就是負載平衡（Load Balancing）登場的地方。

客戶端負載平衡

buildmoat.org

客戶端自己決定要跟哪台伺服器溝通。通常這涉及客戶端向一個服務登錄中心（service registry）請求可用伺服器的列表，然後直接向其中一台發出請求。客戶端需要定期輪詢或接收推送，以知道何時有變化。

客戶端負載平衡可以非常快速高效。由於客戶端自己做決定，它可以選擇最快的伺服器而不增加任何額外延遲。我們只需要定期同步伺服器列表，而不需要在每個請求上都多走一個網路跳點。

例子：Redis Cluster

Redis Cluster 是個好例子。Cluster 的節點之間互相維護一個 gossip 協定，分享叢集資訊。每個節點知道所有其他節點！連線到 Redis Cluster 時，客戶端向任意一個節點請求叢集參與節點和它們負責的分片資訊。讀寫資料時，客戶端雜湊 key 決定要發到哪個分片，然後用本地取得的節點資訊決定要跟哪個節點溝通。如果你發請求給了錯誤的節點，Redis 會友善地回傳 `MOVED` 回應告訴你找錯地方了。

例子：DNS

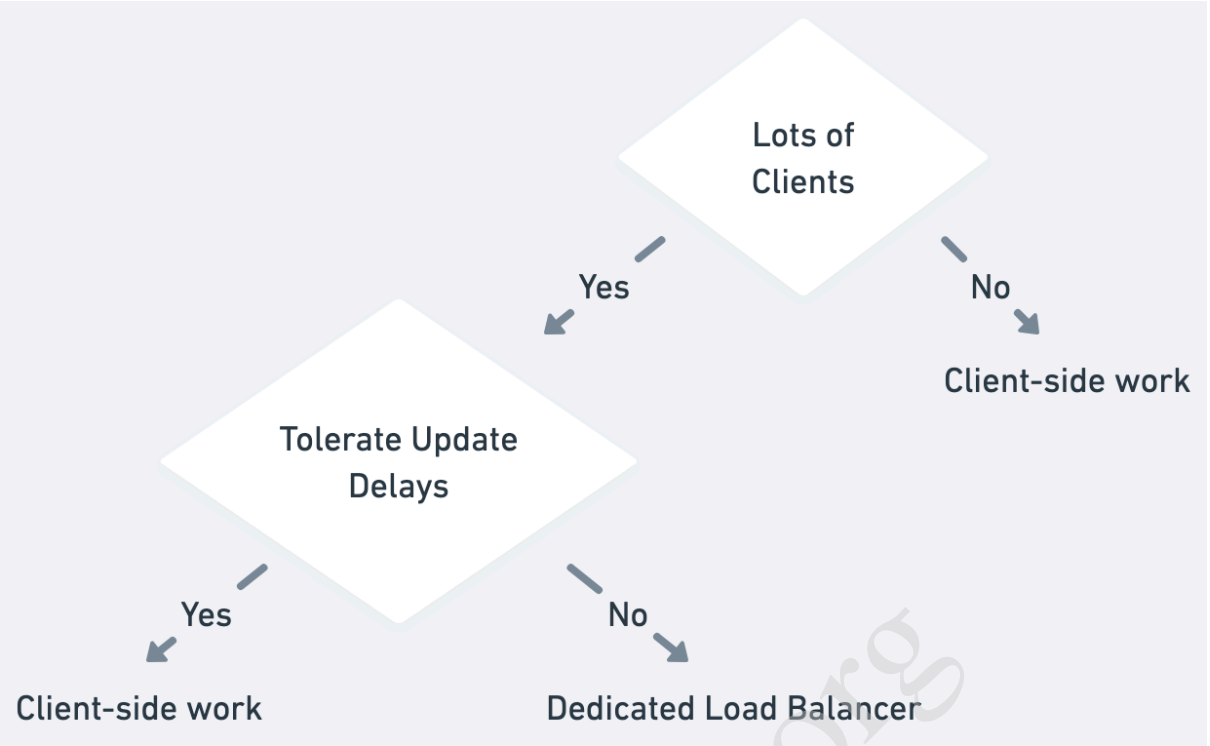
DNS 也是客戶端負載平衡的一個例子。當你向 example.com 這樣的域名發出請求，DNS 解析器會回傳一個輪換過的 IP 位址列表。每個新請求會得到不同順序的 IP 位址（甚至是完全不同的集合）。由於每個客戶端得到不同順序的 IP 位址，它們也會打到不同的伺服器。

這個 DNS 行為也是你避免負載平衡器成為單點故障的方式——設定兩個負載平衡器（在不同的資料中心或區域），用 DNS 輪換。如果一個掛掉，客戶端會自動開始嘗試另一個。

什麼時候用：客戶端負載平衡在兩種情況下效果很好：(1) 我們有少量我們控制的客戶端（例如 Redis Cluster 客戶端，或 gRPC 的客戶端負載平衡用於內部服務），或者 (2) 我們有大量客戶端但可以容忍緩慢的更新（例如 DNS）。

在面試中，客戶端負載平衡對內部微服務效果極好（實際上已內建在 gRPC 裡）。對所有其他使用場景，我們需要專用的負載平衡器。

buildmoa.org



專用負載平衡器

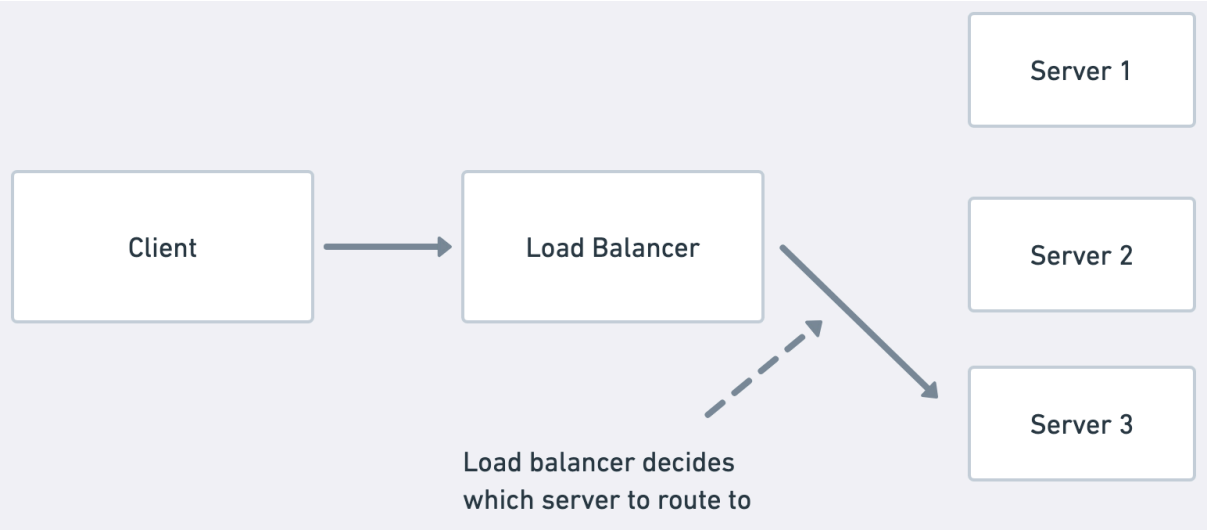
我們可能不希望客戶端必須刷新伺服器列表，甚至不知道後端存在多台伺服器。或者我們可能有大量無法控制的客戶端，但需要快速獲取更新。

在這些情況下，我們使用**專用負載平衡器**：一個位於客戶端和後端伺服器之間的伺服器或硬體裝置，負責決定把請求發到哪台伺服器。

使用專用負載平衡器意味著每個請求都需要額外的一跳：先到負載平衡器，再到實際服務請求的伺服器。但作為交換，我們獲得了對伺服器列表的快速更新和對請求路由方式的細粒度控制。

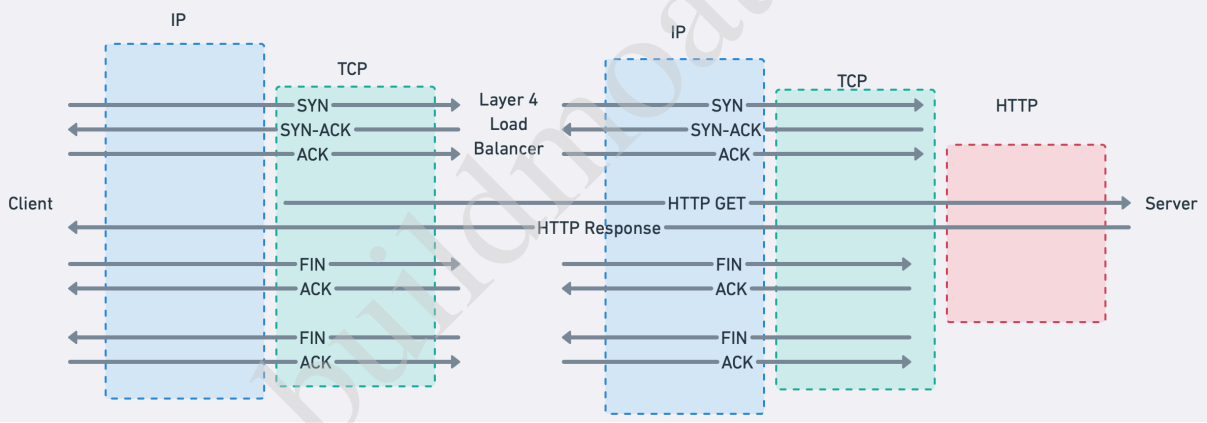
負載平衡器可以在協定堆疊的不同層次上運作，你的選擇部分取決於應用需要什麼。

buildmoat.org



L4 負載平衡器

L4 負載平衡器在傳輸層（TCP/UDP）運作。它根據 IP 位址和 port 等網路資訊做路由決策，不查看封包的實際內容。L4 負載平衡器的效果，就像你隨機選了一台後端伺服器，然後 TCP 連線直接在客戶端和那台伺服器之間建立一樣。



L4 負載平衡器的關鍵特性：

- 維護客戶端和伺服器之間的持久 TCP 連線
- 由於封包檢查最少，速度快且高效
- 無法根據應用層資料做路由決策
- 通常在原始效能是優先考量時使用

如果客戶端透過 L4 負載平衡器建立了 TCP 連線，同一台伺服器將處理該 TCP 工作階段中的所有後續請求。這讓 L4 負載平衡器特別適合需要持久連線的協定，例如 WebSocket 連線。

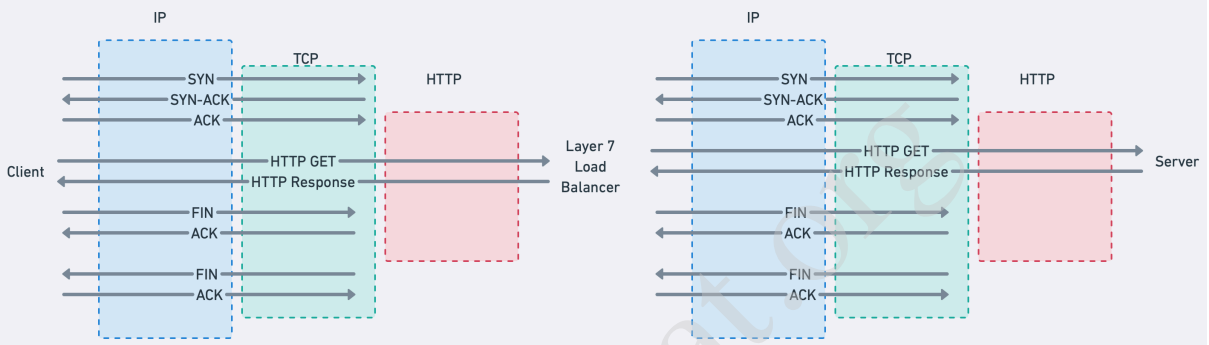
buildmoat.org

什麼時候用：L4 負載平衡器非常適合 WebSocket 連線和其他需要持久連線的協定。如果你在面試中使用 WebSocket，你可能需要 L4 負載平衡器。其他所有情況，L7 負載平衡器可能更合適。

L7 負載平衡器

L7 負載平衡器在應用層運作，理解 HTTP 等協定。它可以檢查每個請求的實際內容，做出更智慧的路由決策。

和 L4 不同，連線層的細節不是那麼重要。L7 負載平衡器接收一個應用層請求（例如 HTTP GET），然後把這個請求轉發給適當的後端伺服器。



L7 負載平衡器的關鍵特性：

- 終止傳入的連線，並向後端伺服器建立新的連線
- 可以根據請求內容（URL、headers、cookies 等）路由
- 由於封包檢查，CPU 消耗更多
- 提供更多靈活性和功能
- 更適合基於 HTTP 的流量

例如，L7 負載平衡器可以把所有 API 請求路由到一組伺服器，同時把網頁請求發到另一組（提供類似 API Gateway 的功能），或者確保特定用戶的所有請求都根據 cookie 發到同一台伺服器。

什麼時候用：L7 負載平衡器非常適合基於 HTTP 的流量，涵蓋了我們討論過的除 WebSocket 之外的所有協定。

健康檢查和容錯

負載平衡器不只負責分配負載和流量，也負責監控後端伺服器的健康狀況。如果一台伺服器斷電或崩潰，負載平衡器會停止向它路由流量，直到它恢復。

buildmoat.io

這種自動故障切換能力是負載平衡器對高可用性至關重要的原因——它可以在沒有用戶介入的情況下偵測故障並繞道而行。

健康檢查的常見方式：

- **TCP 健康檢查**：簡單且高效，檢查伺服器是否接受新連線
- **L7 健康檢查**：向伺服器發出 HTTP 請求，確認回應是成功的（例如 200 狀態碼）

負載平衡演算法

專用負載平衡器的另一個好處是我們對分配流量的演算法有更多選擇：

- **輪詢（Round Robin）**：請求依序分配到各伺服器
- **隨機（Random）**：請求隨機分配到各伺服器
- **最少連線（Least Connections）**：請求發到當前活躍連線最少的伺服器
- **最快回應（Least Response Time）**：請求發到回應時間最快的伺服器
- **IP 雜湊（IP Hash）**：由客戶端 IP 決定哪台伺服器接收請求（適合工作階段持久性）

通常輪詢或隨機演算法是合適的，特別是對無狀態應用。當有新伺服器加入負載平衡器時，這些演算法會自然地開始向它分配流量，不需要特殊設定。

對需要持久連線的服務（例如提供 SSE 或 WebSocket 連線的服務），使用**最少連線**是個好主意，因為它避免了單台伺服器逐漸積累所有活躍連線的情況。

實際的實作選擇

常見的負載平衡器實作：

- **硬體負載平衡器**：F5 Networks BIG-IP 等物理裝置，能擴展到每秒幾億個請求
- **軟體負載平衡器**：HAProxy、NGINX、Envoy
- **雲端負載平衡器**：AWS ELB/ALB/NLB、Google Cloud Load Balancing、Azure Load Balancer

擴展負載平衡器本身幾乎不在軟體工程系統設計面試的範圍內，但如果你發現負載平衡器吞吐量很大——提到硬體負載平衡器是個好的出路。

常見的深挖問題和挑戰

我們已經有了協定、能夠平衡負載、處理持久連線和維護高可用性。還有什麼需要知道的？

雖然某些網路面向可以被面試官假設，但其他面向是深挖問題的好素材。

buildmeat.org

區域化和延遲

對於全球服務，你的伺服器通常分散在世界各地。常見的模式是在單一區域有多個資料中心（Amazon 稱之為「可用區域 Availability Zones」），這樣一棟樓的線路斷掉不會讓整個服務癱瘓，然後在全球各城市複製這個模型。

但這樣的全球部署帶來了新的網路挑戰。客戶端和伺服器之間的物理距離顯著影響網路延遲——光速的限制意味著從紐約到倫敦的請求，延遲永遠比到附近伺服器高（小於 1ms vs 超過 80ms）。

光在光纖電纜中的傳播速度約為真空光速的 2/3，大約每秒 200,000 公里。這意味著紐約到倫敦的往返（約 5,600 公里），僅由信號傳播的物理特性就有理論最低延遲約 56ms，還沒算任何處理時間。

要解決這個問題，我們需要回到**資料局部性**原則：當資料盡可能靠近需要它的計算時，效能最高。

內容傳遞網路（CDN）

最常見的降低延遲策略是使用 CDN。CDN 是策略性分佈在全球的伺服器網路，通常在數百甚至數千個城市都有伺服器（稱為「邊緣位置 edge location」）。如果邊緣伺服器能回答用戶的請求，用戶就能獲得極快的回應時間——資料就在附近！

這只有透過**快取**才能實現。如果我們的資料變動不大，或不需要頻繁更新，我們可以在邊緣伺服器快取它。這對靜態內容（圖片、影片和其他資源）特別有效。

在面試中，當我們有可快取性很高且需要從全球各地查詢的資料時，就會頻繁地看到 CDN 的使用。

區域分片（Regional Partitioning）

另一個處理區域化的常見策略是區域分片。如果我們在單一區域有大量用戶，我們可以按區域分片資料，讓每個區域只有與它相關的資料。

以 Uber 為例：我們在一個特定城市叫車。如果我們在邁阿密，我們永遠不會想預約一個目前在紐約的司機。這是個重要的洞察！

我們可以把附近的城市捆綁成一個本地區域（例如「美國東北」或「美國西南」），每個區域在那個地理位置有自己的資料庫（可能在紐約和洛杉磯設立資料中心）。處理請求的伺服器和它們需要查詢的資料庫共址（co-located）。當用戶想叫車或查看狀態時，查詢可以由區域服務（快速）回答，而那些區域服務可以使用本地資料庫處理查詢（非常快速）。

處理故障和失敗模式

分散式系統最危險的假設之一是「網路是可靠的」。 永遠要在預期網路呼叫會失敗、被延遲或返回意外結果的前提下進行設計。

buildneat.org

逾時和重試加退避（Timeouts and Retries with Backoff）

處理故障最基本的衛生習慣是使用逾時和重試。如果我們預期一個請求需要一定的時間，就設定逾時，超時後放棄並重試。

重試是處理暫時故障的好策略。如果伺服器暫時變慢，重試請求可能就會成功。這裡的關鍵是 API 需要是冪等的，這樣重試相同的請求不會造成問題。

但重試可能是把雙刃劍。如果有大量重試，我們可能會反覆重試一個注定會失敗的請求，反而讓問題更嚴重。這就是為什麼大多數重試策略也包含**退避（backoff）策略**：不是立即重試，而是等待一小段時間再重試，如果還是失敗就等更長時間。這給系統時間恢復，也減少了系統的負載。

退避策略中加入一些隨機性（通常稱為「抖動 jitter」）很重要——讓所有客戶端在同一時間重試沒有任何幫助！最壞的情況是所有失敗的請求同步並反覆在同一時間重試，像打樁機一樣。

在系統設計面試中，面試官通常在找「**指數退避重試（retry with exponential backoff）**」這個魔法短語。在更高級的面試中，你可能被要求詳細說明加入抖動的重要性。

冪等性（Idempotency）

重試很好，除非它們有副作用。想像一個支付系統，如果我們對同一個請求重試多次，就會對用戶重複收費！

這就是為什麼我們需要確保 API 是冪等的。冪等 API 可以被呼叫多次，每次都產生相同的結果。HTTP GET 請求是常見的冪等 API 例子——雖然 GET 請求返回的內容可能改變，但獲取內容的行為本身不會改變系統狀態。

對於寫入資料的場景，常見的做法是在 API 引入一個**冪等鍵（idempotency key）**。例如在支付場景中，如果我們知道用戶每天只會購買一件商品，可以用用戶 ID 和當天日期組成冪等鍵。在伺服器端，我們檢查是否已經處理（或正在處理）帶有該冪等鍵的請求，只處理一次。這讓你避免了重複扣款的問題。

熔斷器（Circuit Breakers）

最後一個常見的深挖話題是如何處理系統中的**級聯故障（cascading failures）**。資深候選人常被問到「如果這個服務掛掉了會怎樣？」有時答案很簡單，但偶爾這會引入新的問題。

如果資料庫完全崩潰，需要一個接一個地重啟實例，而持續湧入的重試和憤怒的用戶可能會讓某個實例根本無法啟動（有時被稱為「驚群效應 thundering herd」）。你無法讓第一個實例起來，所以也沒希望讓整個資料庫重新上線。你被卡住了！

有過 oncall 經驗的工程師會有很多關於級聯故障的故事，這是一個直到在凌晨三點被咬之前都不會被注意到的常見問題。

buildmeat.org

熔斷器是系統設計中一個直接影響網路通訊的關鍵模式。每個 service 內部會維護一個 Circuit Breaker（通常是一個 library，例如 Java 的 Resilience4j、Go 的 gobreaker），自動監控對外呼叫的健康狀態。它的運作方式：

1. **Closed（正常）**：熔斷器監控對外部服務呼叫的失敗情況
2. 當失敗次數超過閾值（例如連續失敗 5 次，或 10 秒內失敗率 > 50%），電路「跳閘（trips）」到**開路 (Open)** 狀態
3. 在 Open 狀態下，請求**立即失敗**而不嘗試實際呼叫，避免繼續壓垮下游
4. 在逾時期後（例如 30 秒），電路自動轉變到**半開路（Half-Open）**狀態
5. Half-Open 狀態下放行一個測試請求：成功則閉合電路恢復正常，失敗則退回 Open 狀態繼續等待

```
Closed（正常運作，監控失敗次數）
  ↓ 失敗超過閾值
Open（拒絕所有請求，等待逾時）
  ↓ 逾時到期（如 30 秒）
Half-Open（放一個測試請求）
  ├── 成功 → 回到 Closed（恢復正常）
  └── 失敗 → 回到 Open（繼續等待）
```

整個過程是自動化的，不需要人工介入。

熔斷器提供了許多優勢：

- **快速失敗**：迅速拒絕對失敗服務的請求，而不是等待逾時
- **減少負載**：防止用更多請求壓垮已經掙扎中的服務
- **自我修復**：在沒有全部流量的情況下自動測試恢復
- **改善用戶體驗**：提供快速的降級回應而不是懸掛的 UI
- **系統穩定性**：防止一個服務的故障影響整個系統

什麼時候用：當面試官深挖可靠性、故障模式或災難恢復時，熔斷器是個很好的回應。能夠提及熔斷器並把它應用在合適的地方，是展示那種只有在凌晨三點對抗硬體故障、系統就是不肯起來的經歷中才能獲得的知識的好方法。

熔斷器適合的地方：

- 對第三方服務的外部 API 呼叫
- 資料庫連線和查詢

buildmoat.org

- 微服務架構中的服務間通訊
- 可能逾時的資源密集型操作
- 任何可能失敗或變慢的網路呼叫

總結

網路是連接分散式系統所有元件的基礎。雖然這個領域很廣闊，但聚焦在這些關鍵領域將使你為大多數系統設計面試做好準備：

- **理解基礎**：IP 定址、DNS 和 TCP/IP 模型
- **了解你的協定**：TCP vs UDP；HTTP/HTTPS；REST、GraphQL、gRPC；SSE；WebSocket；WebRTC——每種的用途和取捨
- **掌握負載平衡**：客戶端負載平衡（Redis Cluster、DNS）和專用負載平衡器（L4 vs L7）
- **規劃實際問題**：區域化、CDN、重試退避、冪等性、熔斷器

記住，網路決策影響系統的每一個面向，像是從延遲和吞吐量，到可靠性和安全性。在面試中，要根據你所設計的系統的具體需求來論證你的網路選擇。我們提供了一堆合理的預設值，但現實是對大多數問題沒有單一的正確答案，你的面試官想看到你如何思考取捨。

自我測驗

Q1：TCP 和 UDP 最根本的差異是什麼？各自適合什麼場景？

> • **TCP**：面向連線、保證可靠交付和順序、有流量控制和壅塞控制。Header 20-60 bytes。適合幾乎所有應用（HTTP、API、檔案傳輸），是面試的預設選擇。
> • **UDP**：無連線、不保證送達和順序、Header 只有 8 bytes。適合低延遲且可容忍丟包的場景（串流、遊戲、VoIP）。

Q2：以下哪個 HTTP method 不是冪等的？(A) GET (B) PUT (C) DELETE (D) POST

> **答案：(D) POST**。GET、PUT、DELETE 都是冪等的（多次呼叫結果相同），POST 不是。

Q3：REST、GraphQL、gRPC 各自最適合的使用場景是什麼？

> • **REST**：外部 API 的預設選擇，簡單、跨平台、易快取。
> • **GraphQL**：前端需要靈活查詢且資料結構複雜時使用，避免 over-fetching 和

buildmoat.org

> under-fetching。
> • **gRPC**：內部微服務間通訊，二進位序列化（Protocol Buffers），吞吐量約為 JSON 的 10 倍。
>
> 策略：**內部 API 用 gRPC，外部 API 用 REST**。

Q4：L4 和 L7 Load Balancer 的差異是什麼？WebSocket 應該用哪個？

> • **L4 Load Balancer**：在傳輸層運作，根據 IP/Port 路由，不理解 HTTP 內容，速度快、開銷低。維護持久 TCP 連線。
> • **L7 Load Balancer**：在應用層運作，理解 HTTP，可根據 URL、header、cookie 路由，功能更強但 CPU 消耗更高。
>
> **WebSocket 應該用 L4**，因為 WebSocket 是長連線，L4 可以維持持久的 TCP 連線。

Q5：Circuit Breaker（熔斷器）的三個狀態是什麼？它解決什麼問題？

> 三個狀態：
>
> 1. **Closed**（正常運作，監控失敗次數）
> 2. **Open**（失敗超過閾值，所有請求立即失敗，不嘗試呼叫）
> 3. **Half-Open**（逾時後，嘗試少量請求測試是否恢復）
>
> 解決的問題：**防止級聯故障**——當下游服務不健康時，不斷重試只會讓情況更糟。熔斷器讓請求快速失敗，給下游服務恢復的時間。

Q6：為什麼重試策略需要加入「抖動（jitter）」？

> 如果大量客戶端在同一時間同步重試，會形成突刺流量（類似 thundering herd），反而讓已經掙扎的服務更加不堪負荷。加入隨機的抖動讓重試時間錯開，分散負載，避免所有客戶端同時重試。

buildmoat.org